

Trainer Guide: Spring Boot Session 1 - Hands-on Lab

A walk-through script for `springboot-session1-lab.html`. Lead the lab and explain every step out loud, even if you just dictate from this guide.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: `springboot-session1-lab.html`

Each lab has an ON SCREEN reminder and a SAY / WALK THROUGH section you can read out as you lead. EMPHASISE and DELIVERY TIP notes are for you only.

How to run this lab

Lab flow & the sticky progress bar

ON SCREEN

The lab page: a blue hero header, a sticky progress bar at the top, and expandable lab sections. Each code block has a Copy button.

SAY / WALK THROUGH

This is our first hands-on session, so let's set the rhythm. The page has a progress bar pinned at the top that fills as you tick checkboxes - it's a nice way to see how far along the room is. Each lab section expands when you click its heading, and every code block has a Copy button, so nobody needs to retype anything. We'll build one tiny application together: a greeting endpoint, backed by a service. It's deliberately small - the goal isn't a big app, it's to see the whole Spring Boot machine work end to end, and to recognise the pieces we covered in the theory.

DELIVERY TIP

Have your own project already created and running before the session, so if someone's setup stalls you can keep the group moving by demoing on your screen.

Pre-requisites Check

ON SCREEN

A checklist of pre-requisites - JDK 17+, an IDE, and internet access for Maven to download dependencies.

SAY / WALK THROUGH

Before we touch code, let's make sure everyone's ready. You need a JDK, version 17 or higher - quickly check with 'java dash version'. You need an IDE - IntelliJ or Eclipse is fine. And you need internet access for the first run, because Maven downloads the dependencies the first time. Tick each item as you confirm it. When all the pre-requisite boxes are green, the 'ready to begin' badge lights up at the top. Take thirty seconds now - raise a hand if 'java dash version' shows anything below 17.

The labs

Lab 1 - Create & Run Your First Spring Boot Project

ON SCREEN

Steps to generate a project on start.spring.io, import it, and run the main class. An expected 'Tomcat started on port 8080' console line.

SAY / WALK THROUGH

Lab one - we create the project from scratch. Go to start dot spring dot io. Choose Maven, Java, the latest stable Spring Boot. Set the Group to com dot tcs dot training and the Artifact to springboot-demo. Packaging Jar, Java 17. For dependencies, add just one: Spring Web. Click Generate, and it downloads a ZIP. Unzip it and import it into your IDE as a Maven project - give it a moment to download dependencies. Then find the main class, SpringbootDemoApplication, and run it like any Java program. Watch the console. You're looking for two lines: 'Tomcat started on port 8080' and 'Started SpringbootDemoApplication'. When you see those, your app is live - even though we haven't written a single line yet. That's auto-configuration and the embedded server from the theory, working for real.

EMPHASISE

Pause on that moment - a running web server with zero code written. Ask the room: 'notice we never installed Tomcat?' That lands the embedded-server idea far better than a slide did.

Lab 2 - Explore the Project Structure

ON SCREEN

A guided tour of pom.xml, the main class, and application.properties, plus a table of the folder layout.

SAY / WALK THROUGH

Lab two is reading, not coding - it builds familiarity with what Boot generated. Open three files with me. First, pom dot xml. Find three things: the parent block - spring-boot-starter-parent, which manages all the versions; the dependencies section - just spring-boot-starter-web; and the build plugin - spring-boot-maven-plugin, which packages the runnable JAR. Second, the main class. Spot the at-SpringBootApplication annotation and the SpringApplication dot run call inside main. That one line boots everything. Third, application dot properties in src-main-resources. It's empty - and that's perfectly fine. Spring Boot's defaults are doing all the work. The folder table on screen shows the rest of the layout.

EMPHASISE

Reinforce the package rule here, because it prevents the number-one beginner bug: the main class sits in com dot tcs dot training, and component scanning only reaches that package and below - so controllers must go in a sub-package like com dot tcs dot training dot controller.

Lab 3 - Your First REST Controller

ON SCREEN

Steps to create a controller package and HelloController with a @GetMapping('/hello'), then hit it in the browser. A 'try it yourself' with extra endpoints.

SAY / WALK THROUGH

Now we write code. Create a package `com dot tcs dot training dot controller` - note, a sub-package of the main class. Inside it, create `HelloController`. Add `@RestController` on the class, and a method with `@GetMapping slash hello` that returns the string 'Hello from Spring Boot'. Use the Copy button so we all have it instantly. Run the app and open `localhost colon 8080 slash hello` in your browser. You should see the exact text come back. Now trace what just happened, because it's the request flow from the theory, live: the browser sent `GET slash hello`, the embedded Tomcat handed it to Spring's `DispatcherServlet`, which routed it to your `sayHello` method, and the method's return value became the response body.

DELIVERY TIP

Do the 'try it yourself' together if there's time - add a `slash welcome` endpoint and a `slash hello slash name` endpoint using `@PathVariable`. Seeing them add their own endpoint cements that mapping is just URL-to-method.

Lab 4 - Dependency Injection in Action

ON SCREEN

Steps to create a `@Service GreetingService` and inject it into the controller with `@Autowired`. An observation task: remove `@Service` and watch startup fail.

SAY / WALK THROUGH

Lab four is the heart of the session - dependency injection. Create a service package and a `GreetingService` class. Mark it with `@Service`, and give it a method `getGreeting` that returns a message. Then, back in `HelloController`, add a `GreetingService` field with `@Autowired`, and have `sayHello` return `greetingService dot getGreeting`. Re-run and reload `slash hello` - it now returns the service's message. Here's the key point: you never wrote 'new `GreetingService`'. Spring created the bean, stored it in its IoC container, and handed it to the controller. That's dependency injection - the thing that makes Spring apps so easy to test and change.

EMPHASISE

Do the observation experiment live - it's the 'aha' moment. Remove `@Service` from `GreetingService` and re-run. The app fails to start with 'a bean of type `GreetingService` could not be found'. That failure teaches more than any slide: no `@Service` means no managed bean, so nothing to inject. Then put it back.

Bonus - Return JSON instead of plain text

ON SCREEN

Steps to create a `Greeting` record and an endpoint returning it, with the expected JSON output.

SAY / WALK THROUGH

If we have time, here's a bonus that previews Session 2. So far we've returned a plain string. Now create a simple record - Greeting, with a message and a from field. Add an endpoint, slash api slash greeting, that returns a new Greeting object. Run it and open that URL. Notice what comes back: not text, but JSON - a proper object with keys and values. You wrote no conversion code. Spring Boot used the Jackson library, which came in with the web starter, to automatically turn your Java object into JSON. That automatic serialization is exactly what we build a whole API on in Session 2.

Reference material & close

Reference sections - troubleshooting, glossary, self-check

ON SCREEN

Three reference sections at the bottom: a troubleshooting table, a plain-English glossary, and six self-check questions.

SAY / WALK THROUGH

The bottom of the sheet has three things to point people to rather than read aloud. The troubleshooting table covers the usual suspects - a 404 from a misplaced controller, 'port 8080 already in use', the missing at-Service error, and changes not showing because the app wasn't restarted. The glossary defines every term they heard today in one plain-English line - bean, IoC container, DI, starter, and so on. And the self-check has six questions; if they can answer all six, they're ready for Session 2. Encourage them to use the troubleshooting table as their first stop whenever something breaks - it'll save you and them a lot of time.

Wrap-up & what's next

ON SCREEN

The 'Coming up in Session 2' box and the quick reference card.

SAY / WALK THROUGH

Let's wrap. Today you created a Spring Boot project, ran it with an embedded server, wrote your first REST controller, and used dependency injection to wire a service into it - the whole machine, end to end. The quick reference card collects every annotation, property, and Maven command from today in one place - a handy revision sheet. And in Session 2 we connect this app to a database and build full CRUD with Spring Data JPA. Bring this same springboot-demo project - we build straight on top of it. Great work, everyone.